

Wollo University, Kombolcha Institute of Technology
College of Informatics, Software Engineering department
Parallel computing (Elective II) Course outline

1. Course Identification

Item	Description
Course Title	Parallel Computing
Course Code	SEngc652
Program	B.Sc. Software Engineering
Course Instructor	Tilahun Ayalew, (tilahun.ayalew@wu.edu.et)
Level	Undergraduate
Year/Semester	V/II
ECTS	5 (2+3)
Total Contact Hours	5 Hours/Week
Prerequisites	Data Structures and Algorithms, Computer Architecture, Operating Systems, Programming II
P r o g r a m m i n g Languages	C/C++, Java and/or Python; C/C++ recommended for OpenMP/MPI
Laboratory	Required
Course Type	Elective

2. Course Description

Parallel Computing introduces the fundamental principles, architectures, programming models, algorithms, performance considerations, and software engineering techniques required to design and implement software systems that execute multiple computations concurrently.

The course develops students' ability to identify computational problems that can benefit from parallelization, analyze dependencies and computational complexity, decompose problems into parallel tasks, select appropriate parallel architectures and programming models, implement parallel algorithms, synchronize concurrent activities, manage communication and shared resources, and evaluate the performance, scalability, reliability, and efficiency of parallel software.

The course covers **parallel architectures, concurrency, task and data parallelism, shared-memory programming, distributed-memory programming, message passing, GPU and accelerator computing, parallel algorithms, synchronization, load balancing, performance measurement, scalability, debugging, testing, and optimization.**

Practical laboratory activities emphasize implementation and experimentation using technologies such as **POSIX threads (Pthreads), OpenMP, MPI, CUDA or equivalent GPU frameworks**, depending on available laboratory infrastructure.

Students complete a practical parallel-computing project in which they transform a sequential application into a parallel implementation, measure performance, analyze speedup and efficiency, identify bottlenecks, and document the engineering decisions and experimental results.

3. Course Rationale

Modern software systems increasingly operate on multicore CPUs, many-core processors, GPUs, cloud infrastructure, clusters, and heterogeneous computing platforms. Consequently, software engineers need to understand not only how to write correct sequential programs but also how to design software capable of exploiting available computational parallelism.

Parallel computing is particularly relevant to:

- Scientific computing
- Artificial intelligence and machine learning
- Big-data processing
- Image and video processing
- Computer graphics
- Simulation and modeling
- Cryptography
- Optimization
- Computer vision
- Cloud computing
- High-performance computing
- Large-scale data analytics
- Engineering applications
- Real-time systems

The course therefore connects **computer architecture, algorithms, operating systems, programming, software design, performance engineering, and system-level thinking.**

4. General Course Goal

The general goal of the course is to enable students to **analyze, design, implement, test, and evaluate parallel software solutions using appropriate parallel architectures, algorithms, programming models, and performance-analysis techniques.**

5. Specific Course Objectives

By the end of the course, students should be able to:

1. Explain fundamental concepts of parallel and concurrent computing.
 2. Differentiate parallel, concurrent, distributed, and sequential computation.
 3. Explain different parallel computer architectures.
 4. Identify opportunities for parallelization in sequential algorithms.
 5. Decompose computational problems into independent parallel tasks.
 6. Analyze data dependencies and synchronization requirements.
 7. Apply shared-memory programming techniques.
 8. Develop parallel programs using OpenMP or equivalent frameworks.
 9. Explain and implement message-passing programs using MPI.
 10. Apply synchronization and communication mechanisms correctly.
 11. Design and analyze parallel algorithms.
 12. Explain GPU and heterogeneous computing principles.
 13. Develop basic GPU-based parallel programs.
 14. Measure execution time, speedup, efficiency, scalability, and throughput.
 15. Identify parallel performance bottlenecks.
 16. Apply load-balancing techniques.
 17. Test and debug parallel applications.
 18. Evaluate correctness and race conditions.
 19. Compare sequential and parallel implementations experimentally.
 20. Develop a complete parallel-computing application as a team.
-

6. Course Learning Outcomes — CLOs

At the completion of the course, students will be able to:

CLO	Course Learning Outcome	Bloom's Level
CLO1	Explain fundamental concepts, architectures, models, and terminology of parallel computing.	Understand
CLO2	Analyze sequential algorithms and identify opportunities and limitations for parallel execution.	Analyze
CLO3	Decompose computational problems into tasks and data structures suitable for parallel execution.	Analyze
CLO4	Design parallel algorithms considering dependencies, communication, synchronization, and load balancing.	Create
CLO5	Implement shared-memory parallel programs using appropriate programming frameworks.	Apply
CLO6	Implement distributed-memory/message-passing programs using MPI or an equivalent framework.	Apply
CLO7	Develop basic GPU/accelerator-based solutions for computationally intensive problems.	Create
CLO8	Test, debug, and verify parallel programs, including detection of race conditions and synchronization errors.	Evaluate
CLO9	Measure and evaluate speedup, efficiency, scalability, throughput, and execution performance.	Evaluate
CLO10	Select appropriate parallel computing technologies for a given software-engineering problem.	Evaluate
CLO11	Work effectively in teams to develop and document a parallel software solution.	Create

CLO Course Learning Outcome**Bloom's****Level**

CLO1 Communicate experimental results and justify parallel design decisions
2 using technical evidence.

Evaluate

7. Relationship to OBE/ABET Student Outcomes

The course contributes particularly strongly to the following ABET-style outcomes:

Student Outcome Contribution of Parallel Computing

SO1 Analyze complex computing/software problems and apply computing principles to develop solutions.

SO2 Design, implement, and evaluate computing-based solutions against requirements.

SO3 Communicate technical findings through reports, presentations, and demonstrations.

SO4 Recognize professional, ethical, security, and responsible-computing considerations.

SO5 Function effectively as a member or leader of a software-development team.

SO6 / discipline-specific Apply computing theory, programming, algorithms, architecture, and software-development fundamentals.

ABET's current computing criteria define student outcomes around problem analysis, solution design/implementation/evaluation, communication, professional responsibility, and teamwork; the computer-science criteria additionally call for applying computer-science theory and software-development fundamentals.

CHAPTER 1 — Introduction to Parallel and Concurrent Computing

1.1 Introduction

- Definition of parallel computing
- Definition of concurrent computing
- Sequential versus parallel computation
- Parallelism versus concurrency
- Distributed versus parallel computing
- High-performance computing
- Cloud and heterogeneous computing

1.2 Motivation for Parallel Computing

- Increasing computational requirements
- Multicore processors
- Many-core processors
- GPU computing
- Large-scale data processing
- Scientific and engineering simulations
- AI and machine learning
- Real-time applications

1.3 Fundamental Concepts

- | | |
|-----------|-----------------|
| • Process | • Processor |
| • Thread | • Node |
| • Task | • Cluster |
| • Core | • Shared memory |

- Distributed memory
- Synchronization

1.4 Types of Parallelism

- Bit-level parallelism
- Instruction-level parallelism
- Data-level parallelism

1.5 Parallel Computing Models

- Shared-memory model
- Distributed-memory model
- Message-passing model
- Data-parallel model

1.6 Parallel Architectures

- SISD
- SIMD
- MISD
- MIMD
- Multicore systems

1.7 Parallel Computing Challenges

- Communication overhead
- Synchronization overhead
- Race conditions
- Deadlock

- Communication

- Task-level parallelism
- Pipeline parallelism
- Functional parallelism
- Task-parallel model
- Hybrid model
- GPU programming model

- Many-core systems
- Clusters
- GPU architectures
- Heterogeneous systems

- Starvation
- Load imbalance
- Memory contention
- Scalability limitations

Laboratory 1

Lab: Introduction to Parallel Execution

Students implement a computational task in sequential and multithreaded versions.

Practical exercises:

1. Sequential summation of an array.
2. Multithreaded summation.
3. Compare execution times.
4. Increase input size.

5. Calculate speedup.

7. Discuss limitations.

6. Calculate efficiency.

CHAPTER 2 — Parallel Algorithm Design and Decomposition

2.1 Sequential versus Parallel Algorithms

2.2 Identifying Parallelism

- Independent operations
- Independent data
- Independent tasks
- Dependency analysis

2.3 Problem Decomposition

- Domain decomposition
- Data decomposition
- Functional decomposition
- Task decomposition
- Recursive decomposition

2.4 Dependency Analysis

- Read-after-read
- Read-after-write
- Write-after-read
- Write-after-write
- Dependency graphs

2.5 Granularity

- Fine-grained parallelism
- Medium-grained parallelism
- Coarse-grained parallelism
- Granularity versus overhead

2.6 Parallel Algorithm Patterns

- Map
- Pipeline
- Reduce
- Producer-consumer
- Scan
- Master-worker
- Divide and conquer
- Fork-join

2.7 Parallel Complexity

- Work
- Parallel time
- Span
- Sequential time

- Communication complexity

2.8 Amdahl's Law

Students calculate the theoretical maximum speedup resulting from parallelization.

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Where:

- PP = parallel portion
- NN = number of processors
- S(N)S(N) = speedup

2.9 Gustafson's Law

Students compare fixed-size and scaled-size performance models.

2.10 Load Balancing

- Static scheduling
- Dynamic scheduling
- Guided scheduling
- Work stealing
- Partitioning

Laboratory 2

Lab: Parallel Algorithm Decomposition

Implement and compare:

1. Parallel vector addition
2. Parallel matrix addition
3. Matrix multiplication
4. Parallel maximum/minimum search
5. Parallel array reduction
6. Parallel sorting

Students must produce:

- Algorithm
- Sequential implementation
- Parallel implementation
- Execution-time table
- Speedup graph
- Efficiency analysis
- Discussion

CHAPTER 3 — Shared-Memory Parallel Programming

3.1 Shared-Memory Systems

3.2 Threads and Processes

3.3 Thread Lifecycle

3.4 POSIX Threads

Topics include:

- Thread creation
- Thread termination
- Thread joining
- Thread arguments
- Thread identification
- Thread synchronization

3.5 Thread Synchronization

- Mutex
- Semaphore
- Barrier
- Condition variables
- Read/write locks

3.6 Critical Sections

3.7 Race Conditions

Example:

```
counter++;
```

Students analyze why simultaneous execution can produce incorrect results.

3.8 Deadlocks

- Mutual exclusion
- No preemption
- Hold and wait
- Circular wait

3.9 OpenMP

Topics:

- Parallel regions
- Threads

- parallel
- for
- sections
- single
- master

- critical
- atomic
- barrier
- reduction
- schedule

3.10 OpenMP Scheduling

- Static
- Dynamic
- Guided
- Runtime

3.11 Data Environment

- Shared variables
- Private variables
- Firstprivate
- Lastprivate
- Reduction variables

Laboratory 3

OpenMP/Pthreads Laboratory

Lab 3.1 — Creating Threads

Create 2, 4, 8, and 16 threads.

Lab 3.2 — Parallel Array Processing

Calculate:

- Sum
- Maximum
- Average
- Minimum

Lab 3.3 — Matrix Multiplication

Compare:

Sequential Matrix Multiplication

versus

OpenMP Matrix Multiplication

Lab 3.4 — Race Condition

Create an intentionally incorrect shared counter and demonstrate the race condition.

Lab 3.5 — Synchronization

Correct the problem using:

- Mutex
- Critical section
- Atomic operation

Lab 3.6 — OpenMP Scheduling

Compare static and dynamic scheduling.

CHAPTER 4 — Distributed-Memory and Message-Passing Computing

4.1 Distributed-Memory Architecture

4.2 Distributed Computing Concepts

4.3 Message Passing

4.4 MPI Architecture

4.5 MPI Environment

- MPI initialization
- MPI termination
- Rank
- Size
- Communicator

4.6 Point-to-Point Communication

- Send
- Blocking communication
- Receive
- Non-blocking communication

4.7 Collective Communication

- Broadcast
- Gather
- Scatter
- Reduce

- All-reduce

- Barrier

4.8 MPI Data Distribution

4.9 Communication Costs

4.10 Synchronization

4.11 Deadlock in MPI

4.12 Distributed Matrix Processing

4.13 Distributed Sorting

4.14 Master-Worker Architecture

4.15 Fault and Failure Considerations

Laboratory 4

MPI Laboratory

Students implement:

Exercise 1

MPI Hello World.

Exercise 2

Display:

Process Rank

Number of Processes

Hostname

Exercise 3

Point-to-point communication.

Exercise 4

Broadcast.

Exercise 5

Scatter and Gather.

Exercise 6

Parallel reduction.

Exercise 7

Distributed vector addition.

Exercise 8

Distributed matrix multiplication.

Exercise 9

Master-worker computation.

Exercise 10

Compare communication and computation overhead.

CHAPTER 5 — GPU and Heterogeneous Parallel Computing

5.1 Introduction to GPU Computing

5.2 CPU versus GPU

5.3 GPU Architecture

- Host
- Device
- Streaming multiprocessors
- CUDA cores
- Threads
- Blocks
- Grids

5.4 GPU Programming Model

5.5 CUDA Programming

- Kernel
- Thread
- Block
- Grid

- Device memory

5.6 Memory Hierarchy

- Global memory
- Shared memory
- Registers

5.7 GPU Parallelism

5.8 Memory Transfer

5.9 Kernel Execution

5.10 Thread Organization

5.11 GPU Synchronization

5.12 GPU Performance

- Memory bandwidth
- Occupancy
- Kernel execution time
- Data-transfer overhead

5.13 Heterogeneous Computing

Combining:

CPU + GPU

CPU + Accelerator

5.14 GPU Applications

- Matrix multiplication
- Image processing
- Vector operations
- Machine learning
- Scientific computation
- Cryptographic computation

Laboratory 5

GPU/CUDA Laboratory

Lab 5.1

CUDA Hello World.

Lab 5.2

Vector addition.

Lab 5.3

Vector multiplication.

Lab 5.4

Matrix addition.

Lab 5.5

Matrix multiplication.

Lab 5.6

Image-processing operation.

Lab 5.7

Compare:

CPU execution

vs.

GPU execution

Students calculate:

- Execution time
- Speedup
- Memory-transfer overhead
- Throughput

CHAPTER 6 — Parallel Performance, Testing, Optimization and Applications

6.1 Parallel Performance

6.2 Performance Metrics

Students calculate:

$$Speedup = \frac{T_s}{T_p}$$

$$Efficiency = \frac{Speedup}{P}$$

$$Throughput = \frac{Number\ of\ tasks}{Execution\ Time}$$

6.3 Scalability

- Strong scaling
- Weak scaling
- Parallel scalability
- Problem-size scalability

6.4 Performance Bottlenecks

- CPU bottleneck
- Memory bottleneck
- Communication bottleneck
- Synchronization bottleneck
- I/O bottleneck

6.5 Profiling

Students learn to identify:

- Hotspots

- CPU utilization
- Memory utilization
- Thread utilization
- Synchronization overhead

6.6 Parallel Debugging

- Race detection
- Deadlock detection
- Incorrect synchronization
- Nondeterministic behavior

6.7 Parallel Testing

- Unit testing
- Integration testing
- Performance testing
- Stress testing
- Scalability testing
- Regression testing

6.8 Parallel Software Optimization

- Algorithm optimization
- Data locality
- Cache utilization
- Loop optimization
- Vectorization
- Load balancing
- Communication reduction

6.9 Reliability and Correctness

6.10 Parallel Software Engineering

- Requirements
- Architecture
- Design
- Implementation
- Testing
- Performance engineering
- Documentation
- Maintenance

6.11 Real-World Applications

Students examine parallel computing applications in:

- AI
- Machine learning
- Image processing
- Computer vision
- Big data
- Weather modeling
- Financial modeling
- Medical imaging
- Cryptography
- Simulation
- Search engines
- Scientific computing

Laboratory 6

Parallel Performance Engineering Laboratory

Students select one computational application and perform:

1. Sequential implementation.
2. Parallel algorithm design.
3. Parallel implementation.
4. Testing.
5. Profiling.
6. Performance measurement.
7. Optimization.
8. Scalability testing.
9. Comparison.
10. Technical documentation.

8. Consolidated Laboratory Program

Lab	Laboratory Topic	Major Skill
Lab 1	Sequential vs Parallel Execution	Basic parallel concepts

Lab	Laboratory Topic	Major Skill
Lab 2	Parallel Algorithm Decomposition	Algorithm design
Lab 3	Pthreads	Thread programming
Lab 4	OpenMP	Shared-memory programming
Lab 5	Synchronization	Race/deadlock control
Lab 6	MPI Basics	Distributed computing
Lab 7	MPI Communication	Message passing
Lab 8	Distributed Matrix Processing	Distributed algorithms
Lab 9	CUDA Fundamentals	GPU programming
Lab 10	CUDA Matrix Operations	GPU algorithms
Lab 11	Performance Measurement	Benchmarking
Lab 12	Parallel Optimization	Performance engineering
Lab 13	Parallel Testing	Verification
Lab 14	Integrated Project	Software engineering

9. Major Course Project

Parallel Software Engineering Project

Students work individually or in teams of **3–5 students**.

Each team selects a computationally intensive application.

Possible projects

- | | |
|---------------------------------------|--|
| 1. Parallel image processing system | 7. Parallel machine-learning algorithm |
| 2. Parallel matrix computation system | 8. Parallel cryptographic computation |
| 3. Parallel sorting application | 9. Parallel medical-image processing |
| 4. Parallel search engine | 10. Parallel traffic simulation |
| 5. Parallel numerical simulation | 11. Parallel weather-data processing |
| 6. Parallel data-analysis application | 12. Parallel financial-data analysis |

Project Requirements

The project must include:

- | | |
|-------------------------------|----------------------------|
| 1. Problem definition | 8. Testing |
| 2. Requirements specification | 9. Performance measurement |
| 3. Sequential baseline | 10. Optimization |
| 4. Parallelization analysis | 11. Scalability analysis |
| 5. Algorithm design | 12. Documentation |
| 6. Architecture design | 13. Demonstration |
| 7. Parallel implementation | 14. Final presentation |
-

10. Recommended Teaching and Learning Methods

The course should use an active OBE-oriented teaching approach.

Methods

- | | |
|----------------------------|---------------------------|
| • Lectures | • Project-based learning |
| • Demonstrations | • Team-based learning |
| • Programming laboratories | • Case studies |
| • Problem-based learning | • Algorithm visualization |

- Performance experiments
- Peer discussion
- Technical presentations

The emphasis should be on **"learn → implement → measure → analyze → improve."**

11. Assessment Strategy

An OBE course should assess both knowledge and demonstrable skills rather than relying exclusively on a final examination. ABET's criteria emphasize documented assessment and systematic use of assessment results for continuous improvement.

Recommended Assessment Distribution

Assessment	Weight
Continuous Assessment/Quizzes	10%
Assignments	10%
Laboratory Exercises	20%
Project	20%
Project Presentation/ Demonstration	5%
Final Examination	35%
Total	100%

12. CLO Assessment Matrix

CLO	Quiz	Assignment	Laboratory	Midterm	Project	Final
CLO1	✓		✓	✓		✓
CLO2	✓	✓	✓	✓	✓	✓

CLO	Quiz	Assignment	Lab	Midterm	Project	Final
CLO3		✓	✓	✓	✓	✓
CLO4		✓	✓		✓	✓
CLO5			✓		✓	✓
CLO6			✓		✓	✓
CLO7			✓		✓	✓
CLO8		✓	✓		✓	✓
CLO9		✓	✓		✓	✓
CLO10		✓	✓		✓	✓
CLO11			✓		✓	
CLO12			✓		✓	

13. CLO–Bloom's Taxonomy Mapping

CLO	Remember	Understand	Apply	Analyze	Evaluate	Create
CLO1	✓	✓				
CLO2		✓	✓	✓		
CLO3			✓	✓		✓
CLO4			✓	✓	✓	✓
CLO5			✓			✓
CLO6			✓			✓
CLO7			✓	✓		✓
CLO8			✓	✓	✓	

CLO	Remember	Understand	Apply	Analyze	Evaluate	Create
CLO9			✓	✓	✓	
CLO10				✓	✓	
CLO11			✓		✓	✓
CLO12				✓	✓	✓

14. Weekly Course Schedule

A **15-week semester** can be organized as follows:

Week	Theory	Laboratory/Activity
1	Introduction to Parallel Computing	Environment setup
2	Parallel architectures	Sequential baseline
3	Parallelism and decomposition	Algorithm decomposition
4	Parallel algorithms	Vector/matrix operations
5	Threads and shared memory	Pthreads
6	OpenMP	OpenMP programming
7	Synchronization	Race conditions/deadlocks
8	Midterm	Review
9	Distributed-memory computing	MPI introduction
10	MPI communication	Scatter/Gather/Reduce
11	Distributed algorithms	Matrix computation
12	GPU computing	CUDA fundamentals

Week	Theory	Laboratory/Activity
13	GPU programming	C U D A m a t r i x / i m a g e processing
14	Performance engineering	Profiling/optimization
15	Project presentations	Demonstration and evaluation

15. Laboratory Environment

A suitable laboratory should provide:

Hardware

- Multicore computers
- At least 4 CPU cores recommended
- Adequate RAM
- Optional NVIDIA GPU
- Local network for MPI experimentation

Operating System

Recommended:

- Ubuntu Linux
- Other Linux distributions
- Windows with appropriate development environments

Software

Recommended:

- GCC/G++
- Make/CMake

- Visual Studio Code
- Git
- OpenMP
- MPI/OpenMPI
- Pthreads
- CUDA Toolkit where NVIDIA GPUs are available
- Python
- Jupyter Notebook
- Performance/profiling tools

The exact technology stack can be adjusted according to institutional laboratory resources.

16. Required Student Practical Skills

By the end of the laboratory component, students should be capable of:

Sequential Program



Analyze Algorithm



Identify Parallelism



Decompose Problem



Select Architecture



Select Programming Model



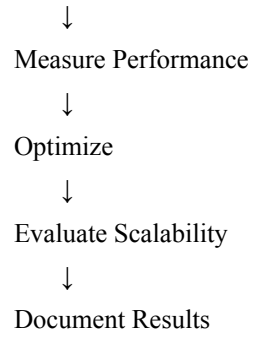
Implement Parallel Algorithm



Synchronize/Communicate



Test and Debug



17.Key Mathematical Concepts

19. Key Mathematical Concepts

Students should apply:

Speedup

$$S_p = \frac{T_1}{T_p}$$

Efficiency

$$E_p = \frac{S_p}{p}$$

Cost

$$C_p = pT_p$$

Amdahl's Law

$$S_p = \frac{1}{(1 - f) + \frac{f}{p}}$$

Gustafson's Law

$$S(p) = p - \alpha(p - 1)$$

where α represents the serial fraction under the scaled workload model.

where α represents the serial fraction under the scaled workload model.

18. Recommended Textbooks and References

1. Barbara Chapman, Gabriele Jost, and Ruud van der Pas, **Using OpenMP: Portable Shared Memory Parallel Programming.**
 2. Peter S. Pacheco, **An Introduction to Parallel Programming.**
 3. Michael J. Quinn, **Parallel Programming in C with MPI and OpenMP.**
 4. Thomas Rauber and Gudula Rünger, **Parallel Programming: For Multicore and Cluster Systems.**
 5. David B. Kirk and Wen-mei W. Hwu, **Programming Massively Parallel Processors.**
 6. Ian Foster, **Designing and Building Parallel Programs.**
 7. Relevant official documentation for **OpenMP, MPI, CUDA, GCC and Linux.**
-

19. Suggested Course Exit-Level Performance

The course should not be considered successfully completed merely because a student can define parallel computing concepts. The student should be able to **demonstrate** a working parallel application and provide evidence that parallelization produces an appropriate computational benefit.

A suitable final performance statement is:

"Given a computationally intensive software problem, the student will be able to analyze the problem, identify exploitable parallelism, design an appropriate parallel algorithm, implement the solution using a suitable parallel programming model, test and debug the

implementation, measure speedup and efficiency, evaluate scalability, and communicate the results using appropriate technical documentation."

This makes the course strongly **outcome-oriented, practical, measurable, and suitable for a Software Engineering curriculum**. It also fits the ABET emphasis on designing and evaluating computing-based solutions, communication, teamwork, and continuous assessment

Recommended course structure at a glance

Component	Coverage
Chapter 1	Parallel and Concurrent Computing Fundamentals
Chapter 2	Parallel Algorithm Design and Decomposition
Chapter 3	Shared-Memory Programming
Chapter 4	Distributed-Memory and MPI Programming
Chapter 5	GPU and Heterogeneous Computing
Chapter 6	Performance, Testing, Optimization and Applications
Laboratory	14 progressive practical exercises
Project	Complete parallel software engineering project
M a i n technologies	Pthreads, OpenMP, MPI, CUDA
Major outcomes	Analyze → Design → Implement → Test → Measure → Optimize → Communicate